



BITS Pilani
K K Birla Goa Campus

INSTRUCTION SET WITH MASM

MICROPROCESSORS & INTERFACING (CS F241)

Dr. Gargi Alavani
Department of CS & IS



String Comparisons

- String instructions are very powerful because they allow the programmer to manipulate large blocks of data with relative ease.
- Block data manipulation occurs with the string instructions MOVS, LODS, STOS.
- Instructions that allow a section of memory to be tested against a constant or against another section of memory:
 - SCAS (string scan)
 - CMPS (string compare)

SCAS (string scan instruction)

- Compares the **AL register** with **a byte block of memory**, the **AX register** with a **word block of memory**, or the EAX register (80386-Core2) with a doubleword block of memory.
- Subtracts memory from AL, AX, or EAX without affecting either the register or the memory location
- SCASB – Byte Comparison
- SCASW – Word Comparison
- SCASD – Double Word Comparison
- Contents of the extra segment memory location addressed by DI is compared with AL, AX, or EAX

SCASB Example : Search part of memory for 00H

```
MOV    DI,OFFSET BLOCK    ;address data
CLD                      ;auto-increment
MOV     CX,100             ;load counter
XOR     AL,AL              ;clear AL
REPNE   SCASB
```

SCASB Example : Skip ASCII-coded spaces

```
CLD                ;auto-increment
MOV    CX, 256     ;load counter
MOV    AL, 20H     ;get space
REPE   SCASB
```

CMPS: String Compare

- The CMPS (compare strings instruction) always compares two sections of memory data as bytes (CMPSB), words (CMPSW), or doublewords (CMPSD)
- The contents of the **data segment memory location addressed by SI** are compared with the contents of the **extra segment memory location addressed by DI**.
- The CMPS instruction increments or decrements both SI and DI.
- CMPS instruction is normally used with either the REPE or REPNE prefix.

CMPS Example

```
MOV    SI,OFFSET LINE    ;address LINE
MOV    DI,OFFSET TABLE  ;address TABLE
CLD                                ;auto-increment
MOV    CX,10                ;load counter
REPE   CMPSB                ;search
```

MASM

- MASM, short for Microsoft Macro Assembler, is a low-level programming language used for programming in assembly language.
- Developed by Microsoft, MASM provides developers with a powerful toolset for writing efficient and optimized code for x86 and x64 architectures.
- Originally introduced in the early 1980s, MASM has since evolved and been updated to support modern computing platforms.
- MASM is widely used in various domains including system programming, device drivers, embedded systems, and performance-critical applications.

Basic Rules

- MASM follows a syntax that is based on mnemonic instructions and directives.
- Instructions are symbolic representations of machine-level operations, while directives are commands to the assembler itself.
- Examples of directives include .MODEL, .DATA, .CODE, etc.
- Comments in MASM start with a semicolon (;) and continue until the end of the line.
- Labels are used to mark specific locations in the code and are followed by a colon (:).
- MASM is not case-insensitive, meaning that upper and lower case letters are treated the same way.

Directives

- **TITLE** line (optional)
 - Contains a brief heading of the program and the disk file name
- **.MODEL** directive
 - Specifies the memory model configuration
- **TINY**: Suitable for small programs that fit within a single code segment and don't require data or stack segments.
- **SMALL**: Suitable for small to moderately sized programs with separate code, data, and stack segments. Code and data segments can be up to 64KB in size.
- **COMPACT, MEDIUM, LARGE, HUGE**: These memory models are suitable for larger programs that require more memory segmentation. They provide increasing levels of memory management and segmentation capabilities.

Directives

.STACK directive

- Tells the assembler to define a runtime stack for the program
- The size of the stack can be optionally specified by this directive
- The runtime stack is required for procedure calls

.DATA directive

- Defines an area in memory for the program data
- The program's variables should be defined under this directive
- Assembler will allocate and initialize the storage of variables

.CODE directive

- Defines the code section of a program containing instructions
- Assembler will place the instructions in the code area in memory

Directives

INCLUDE directive

- Causes the assembler to include code from another file

PROC and **ENDP** directives

- Used to define procedures
- As a convention, you may define ***main*** as the first procedure
- Additional procedures can be defined after ***main***

END directive

- Marks the end of a program
- Identifies the name (***main***) of the program's startup procedure

Data Types

DB (Define Byte):

- Used to declare one or more bytes.

Syntax: `variable_name DB initial_value`

DW (Define Word):

- Used to declare one or more 16-bit words (2 bytes).

Syntax: `variable_name DW initial_value`

DD (Define Doubleword):

- Used to declare one or more 32-bit double words (4 bytes).

Syntax: `variable_name DD initial_value`

Directives

- Indicate how an operand or section of a program is to be processed by the assembler.
- By default the assembler accepts only 8086/8088 instructions, unless a program is preceded by the .686 or .686P directive

MASM Directives

<i>Directive</i>	<i>Function</i>
.286	Selects the 80286 instruction set
.286P	Selects the 80286 protected mode instruction set
.386	Selects the 80386 instruction set
.386P	Selects the 80386 protected mode instruction set
.486	Selects the 80486 instruction set
.486P	Selects the 80486 protected mode instruction set
.586	Selects the Pentium instruction set
.586P	Selects the Pentium protected mode instruction set
.686	Selects the Pentium Pro–Core2 instruction set
.686P	Selects the Pentium Pro–Core2 protected mode instruction set
.287	Selects the 80287 math coprocessor
.387	Selects the 80387 math coprocessor

MASM Directives

<i>Directive</i>	<i>Function</i>
.CODE	Indicates the start of the code segment (models only)
.DATA	Indicates the start of the data segment (models only)
.EXIT	Exits to DOS (models only)
.MODEL	Selects the programming model
.STACK	Selects the start of the stack segment (models only)
.STARTUP	Indicates the starting instruction in a program (models only)
ALIGN n	Align to boundary n (n = 2 for words, n = 4 for doublewords)
ASSUME	Informs the assembler to name each segment (full segments only)
BYTE	Indicates byte-sized as in BYTE PTR
DB	Defines byte(s) (8 bits)
DD	Defines doubleword(s) (32 bits)
DQ	Defines quadwords(s) (64 bits)
DT	Defines ten byte(s) (80 bits)
DUP	Generates duplicates
DW	Define word(s) (16 bits)
DWORD	Indicates doubleword-sized, as in DWORD PTR
END	Ends a program file
ENDM	Ends a MACRO sequence
ENDP	Ends a procedure

MASM Directives

<i>Directive</i>	<i>Function</i>
ENDS	Ends a segment or data structure
EQU	Equates data or a label to a label
FAR	Defines a far pointer, as in FAR PTR
MACRO	Designates the start of a MACRO sequence
NEAR	Defines a near pointer, as in NEAR PTR
OFFSET	Specifies an offset address
ORG	Sets the origin within a segment
OWORD	Indicates octalwords, as in OWORD PTR
PROC	Starts a procedure
PTR	Designates a pointer
QWORD	Indicates quadwords, as in QWORD PTR
SEGMENT	Starts a segment for full segments
STACK	Starts a stack segment for full segments
STRUC	Defines the start of a data structure
USES	Automatically pushes and pops registers
USE16	Uses 16-bit instruction mode
USE32	Uses 32-bit instruction mode
WORD	Indicates word-sized, as in WORD PTR

Storing Data in a Memory Segment

```

LIST_SEG      SEGMENT

DATA1 DB  1,2,3          ;define bytes
        DB  45H          ;hexadecimal
        DB  'A'          ;ASCII
        DB  11110000B    ;binary
DATA2 DW  12,13          ;define words
        DW  LIST1        ;symbolic
        DW  2345H        ;hexadecimal
DATA3 DD  300H           ;define doubleword
        DD  2.123        ;real
        DD  3.34E+12     ;real
LISTA DB  ?             ;reserve 1 byte
LISTB DB  10 DUP(?)     ;reserve 10 bytes

        ALIGN 2          ;set word boundary

LISTC DW  100H DUP(0)    ;reserve 100H words
LISTD DD  22 DUP(?)     ;reserve 22 doublewords

SIXES DB  100 DUP(6)    ;reserve 100 bytes

LIST_SEG      ENDS
    
```

EQU Directive

- EQU equates a numeric, ASCII, or label to another label.

Syntax: identifier EQU expression.

- Example:

TEN	EQU	10
NINE	EQU	9
MOV AL, TEN		
ADD AL, NINE		

- It assigns a fixed value to a symbol, which cannot be changed during program execution.
- Useful for defining constants such as numerical values, memory addresses, or bit masks.

ORG Directive

- The ORG directive specifies the origin (starting address) for a segment in memory.

Syntax: ORG expression

Example: ORG 100h

- It sets the starting address for the subsequent code or data segment.
- Allows organization of program memory by specifying where each segment begins.
- Essential for positioning code or data at specific memory locations.

Model Example

```
.MODEL SMALL          ;select small model
.STACK 100H           ;define stack
.DATA                 ;start data segment

LISTA DB 100 DUP(?)

LISTB DB 100 DUP(?)

.CODE                 ;start code segment

HERE: MOV AX,@DATA     ;load ES and DS
      MOV ES,AX
      MOV DS,AX
      CLD               ;move data
      MOV SI,OFFSET LISTA
      MOV DI,OFFSET LISTB
      MOV CX,100
      REP MOVSB

      .EXIT 0           ;exit to DOS
END HERE
```

Full-Segment Definitions

```

STACK_SEG      SEGMENT      'STACK'
                DW           100H DUP (?)

STACK_SEG      ENDS

DATA_SEG       SEGMENT      'DATA'
LISTA  DB      100 DUP (?)

LISTB  DB      100 DUP (?)

DATA_SEG       ENDS

CODE_SEG       SEGMENT      'CODE'
                ASSUME CS:CODE_SEG, DS:DATA_SEG
                ASSUME SS:STACK_SEG
MAIN  PROC     FAR
        MOV     AX,DATA_SEG           ;load DS and ES
        MOV     ES,AX
        MOV     DS,AX
        CLD                           ;save data
        MOV     SI,OFFSET LISTA
        MOV     DI,OFFSET LISTB
        MOV     CX,100
        REP     MOVSB
        MOV     AH,4CH                ;exit to DOS
        INT     21H
MAIN  ENDP
CODE_SEG      ENDS
                END      MAIN
    
```

Example

MODEL

```

.MODEL TINY
.CODE
.STARTUP

MAIN:
    MOV     AH,6             ;read a key
    MOV     DL,0FFH
    INT     21H
    JE      MAIN             ;if no key typed
    CMP     AL,'@'
    JE      MAIN1            ;if an @ key
    MOV     AH,06H           ;display key (echo)
    MOV     DL,AL
    INT     21H
    JMP     MAIN             ;repeat

MAIN1:

.EXIT             ;exit to DOS
END

```

FULL SEGMENT DEFINITIONS

```

CODE_SEG      SEGMENT 'CODE'
                ASSUME CS:CODE_SEG

MAIN          PROC      FAR

                MOV     AH,06H             ;read a key
                MOV     DL,0FFH
                INT     21H
                JE      MAIN             ;if no key typed
                CMP     AL,'@'
                JE      MAIN1            ;if an @ key
                MOV     AH,06H           ;display key (echo)
                MOV     DL,AL
                INT     21H
                JMP     MAIN             ;repeat

MAIN1:
                MOV     AH,4CH           ;exit to DOS
                INT     21H

MAIN          ENDP
                END      MAIN

```

MASM Example

```
DATAS SEGMENT
    STRING DB 'Hello World! Its 2025!','$'
DATAS ENDS

CODES SEGMENT
    ASSUME CS:CODES,DS:DATAS

START:
    MOV AX,DATAS
    MOV DS,AX

    LEA DX,[STRING]

    MOV AH,9
    INT 21H

    MOV AH,4CH
    INT 21H
CODES ENDS
END START
```

This moves the value 09h into the AH register. This value is the DOS function number for "Display String" using the DOS interrupt 21h

MASM Example

```
DATAS SEGMENT
    STRING DB 'Hello World! Its 2025!','$'
DATAS ENDS
```

```
CODES SEGMENT
    ASSUME CS:CODES,DS:DATAS
```

```
START:
    MOV AX,DATAS
    MOV DS,AX
```

```
    LEA DX,[STRING]
```

```
    MOV AH,09H
    INT 21H
```

```
    MOV AH,4CH
    INT 21H
CODES ENDS
END START
```

This generates a software interrupt (interrupt 21h), which is a DOS interrupt. When AH contains 09h and DS:DX points to a \$-terminated string, this function prints the string to the screen.

MASM Example

```
.model small  
.stack 100h
```

```
.data  
msg      db      'Hello world!$'
```

```
.code  
start:
```

```
    mov     ah, 09h  
    lea     dx, msg  
    int     21h  
    mov     ax, 4C00h ;  
    int     21h
```

This moves the value 4C00h into the AX register. This value is the DOS function number for "Terminate Program" using the DOS interrupt 21h.

```
end start
```

MASM Example

```
.model small
.stack 100h

.data
msg      db      'Hello world!$'

.code
start:

        mov      ah, 09h
        lea      dx, msg
        int      21h
        mov      ax, 4C00h ;
        int      21h
```

This generates another software interrupt (interrupt 21h), this time to terminate the program. When AH contains 4Ch, this function terminates the program.

Running MASM code

- Open Dosbox, mount the drive of MASM folder, go to bin where your .asm is saved.
- Compile your code
masm helloworld.asm
- Linkin your code
link helloworld.org (Click enter three times)
- Execute your code
helloworld.exe

Example 2

```
.model small  
.stack 64  
.data  
a db 02h,02h,02h,02h,02h,02h,02h,02h,02h,02h
```

```
.code  
start: mov ax,@data  
       mov ds,ax  
       mov cl,10  
       lea si,a  
       mov ax,0000h  
again: add al,[si]  
       inc si  
       dec cl  
       jnz again  
       mov cl,0ah  
       div cl  
       mov ah,4ch  
       int 21h  
       end start  
.end
```

Example 3

```
.model small  
.stack 64  
.data  
.code  
start:  
mov al,00h  
mov dl,00h  
mov bl,01h  
mov cl,05h  
again: add al,bl  
mov dl,al  
mov al,bl  
mov bl,dl  
dec cl  
jnz again  
mov ah,4ch  
int 21h  
end start  
.end
```

How to debug in MASM?

```
DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>mount d d:/
Drive D is mounted as local directory d:\

Z:\>d:/

D:\>cd MASM\MASM611\BIN

D:\MASM\MASM611\BIN>debug SIMPLE.EXE
-t

AX=076B BX=0000 CX=0019 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=075A ES=075A SS=076C CS=076A IP=0003  NU UP EI PL NZ NA PO NC
076A:0003 BED8          MOV     DS,AX

-u_
```

How to debug in MASM?

```
DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program: DEBUG
Z:\>d:\
D:\>cd MASM\MASM611\BIN
D:\MASM\MASM611\BIN>debug SIMPLE.EXE
-t
AX=076B BX=0000 CX=0019 DX=0000 SP=0100 BP=0000 SI=0000 DI=0000
DS=075A ES=075A SS=076C CS=076A IP=0003  NU UP EI PL NZ NA PO NC
076A:0003 8ED8          MOV     DS,AX
-u 0003
076A:0003 8ED8          MOV     DS,AX
076A:0005 A00600          MOV     AL,[0006]
076A:0008 8A1E0700          MOV     BL,[0007]
076A:000C 02C3          ADD     AL,BL
076A:000E A20800          MOV     [0008],AL
076A:0011 B44C          MOV     AH,4C
076A:0013 CD21          INT     21
076A:0015 00361B00          ADD     [001B],DH
076A:0019 0000          ADD     [BX+SI],AL
076A:001B 0000          ADD     [BX+SI],AL
076A:001D 0000          ADD     [BX+SI],AL
076A:001F 0000          ADD     [BX+SI],AL
076A:0021 0000          ADD     [BX+SI],AL
```




BITS Pilani
K K Birla Goa Campus

THANK YOU

